

Searching Binary Embeddings

Bitplanes and Prefix Backward Walk

13 September 2026

Contents

1	Introduction	2
1.1	Exa method and end to end pipeline	2
1.2	Idea background	2
1.3	Exploring new methods	3
2	Preliminary calculations	11
2.1	Query-time work	12
2.2	Memory complexity	25
2.3	Recall	26
2.4	Summary	29
3	Experiments	29
3.1	Setup	29
3.2	How the comparison works	31
3.3	Main result	31
3.4	Why is Bitplanes faster?	32
3.5	Does this advantage always happen?	34
3.6	Summary	35
4	Limitations and next steps	35
5	Conclusion	36
	References	36

Abstract

Following Exa’s published work on binary embedding search, this report looks at the stage after nearby clusters have already been selected, where a floating-point query is compared against binary document vectors inside those clusters.

The key observation is that, from the query itself, we already know what the ideal binary document would look like and how costly each bit mismatch is. This raises a simple question: can we use that information to search the selected documents more efficiently than just scoring all of them?

I compare the original full scan with two alternatives: **Bitplanes**, which filters documents using the most important query bits, and **Backward Walk**, which starts from a specific binary prefix and broadens the search backwards. The goal is to see whether the extra search work can save enough query-document dot product operations to reduce latency at the same recall.

1 Introduction

1.1 Exa method and end to end pipeline

Let’s first understand the “original” pipeline that I am trying to improve upon.

Note that the architecture I am referring to here is based on the pipeline described in Exa’s article [1]. I am aware that Exa’s actual production system may have changed or improved since then, but for the purpose of this report, let’s treat the architecture described in that article as the original baseline that we compare against.

Detail: In this pipeline, document embeddings are first shortened using Matryoshka representations, converted into binary signs, and assigned to clusters. At query time, the query is encoded, a subset of nearby clusters is selected, and the documents inside those clusters undergo scoring operations using the floating-point query. The top candidates can then be reranked using the original uncompressed vectors.

The main part that I am trying to improve in this report is the search stage after cluster selection. The two new approaches introduced later, Bitplanes and Backward Walk, replace this step while keeping the rest of the pipeline largely the same (maybe the difference is in the “hyperparameter” used during the experiments).

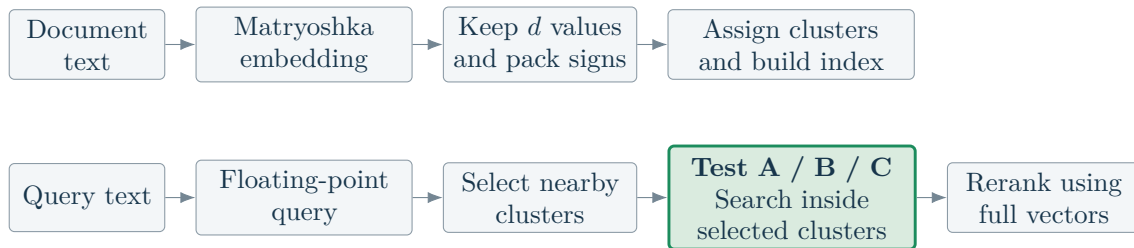


Figure 1: Green marks the part we aim to improve. Method A is the original method (normal scan)

1.2 Idea background

When I took a closer look at the pipeline, one part that particularly stood out to me was the comparison between the floating-point query and the binary-quantized document.

Consider a simple example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3]$$

and suppose a document is represented using binary signs, where 1 means +1 and 0 means −1. For example:

$$\text{Document} = 11000$$

The score value is calculated by taking the dot product between the query values and the document signs. So for 11000, the score value is

$$0.7(+1) + 0.5(+1) + (-0.4)(-1) + (-0.6)(-1) + 0.3(-1) = 1.9.$$

Now, something interesting follows from this. Just by looking at the signs of the query, we already know what the ideal binary document would look like.

For every positive query value, we would prefer the corresponding document bit to be 1, and for every negative query value, we would prefer it to be 0. Therefore, for the query above, the ideal binary pattern is

$$11001.$$

This document would achieve the maximum possible score value:

$$0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5.$$

From here, we can think about every other document in terms of how far it is from this ideal pattern. More importantly, not every mismatch has the same cost. If we mismatch the first coordinate, where $|q_1| = 0.7$, the score value decreases by 1.4. But if we mismatch the last coordinate, where $|q_5| = 0.3$, the score value only decreases by 0.6.

In more formal terms, let

$$Q = \sum_i |q_i|$$

be the score value of the ideal binary pattern. If a document b disagrees with the query sign at some coordinates, define its mismatch penalty as

$$p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|.$$

Then its score value can be written as

$$S(q, b) = Q - 2p. \tag{1}$$

This observation is the main idea behind the approaches explored in this report. Instead of treating every binary document equally and performing a scoring operation for each one, can we use the query itself to guide the search toward documents that are closer to its ideal binary pattern, while avoiding unnecessary query-document dot product operations?

1.3 Exploring new methods

From the previous section, we know two useful things before we even start searching:

We know what the ideal binary document looks like from the signs of the query. We know that some bits matter more than others, based on the magnitude $|q_i|$.

The next question is therefore:

Can we build an index that uses this information directly, instead of treating every document inside the selected clusters equally?

This led me to explore two approaches.

The first, Bitplanes, starts from a large set of candidate documents and progressively narrows it down using the query.

The second, Backward Walk, approaches the problem from the opposite direction: it starts from a very specific binary pattern and progressively relaxes the constraints until enough useful candidates are found (or, fulfill the estimated recall target).

1.3.1 Bitplanes

Now that we know what the ideal document looks like based on the query, one idea is to build a data structure that makes filtering by individual bits very cheap.

Normally, binary documents are stored row by row:

Document ID 2: 10100

Document ID 3: 11001

Document ID 4: 11000

Instead, we can additionally store an inverted representation of the same bits:

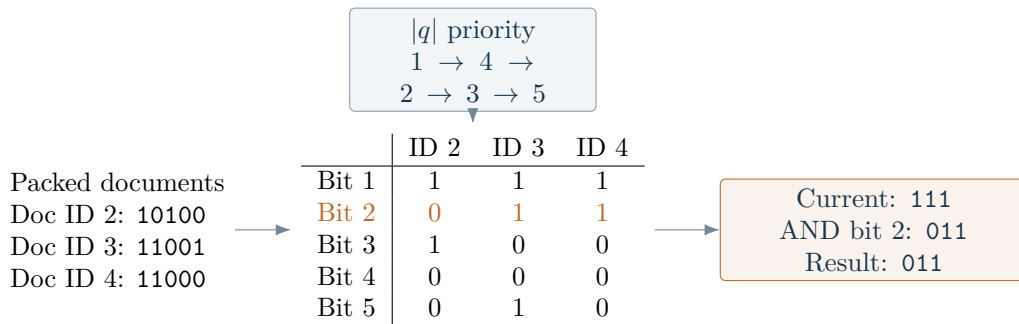


Figure 2: The same document signs, stored in an extra layout for bitwise filtering.

Each row above is what I call a bitplane: one bit position across many documents.

Suppose the query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the magnitude of the query values, we can prioritize the bits approximately as

$$1, 4, 2, 3, 5,$$

because

$$0.7 > 0.6 > 0.5 > 0.4 > 0.3.$$

Now suppose our current candidate mask is

$$111$$

meaning that IDs 2, 3 and 4 are all still possible candidates.

For bit 2, the query prefers 1. The corresponding bitplane is

$$011$$

so keeping the documents that match the query is simply

$$111 \& 011 = 011.$$

We immediately reduce the candidate set to IDs 3 and 4.

This is attractive because the filtering itself consists mostly of bitwise operations, which modern CPUs can perform very efficiently (and maybe can be optimized using SIMD/auto-vectorization by compiler in the future).

A simplified first version of the idea might therefore look like:

```
def full_score(query, code):
    return sum(q if bit else -q for q, bit in zip(query, code))

def greedy_search(query, cluster, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    active = cluster.full_mask
    for bit in order:
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        active &= preferred
        if active.bit_count() <= leaf_size:
            break

    best = TopK(k)
    for row_id, code in cluster.rows_in_mask(active):
        best.add(row_id, full_score(query, code))
    return best.results()
```

However, this simple version has a major problem.

PROBLEM 1: Greedily following the preferred bits can miss the best document

Consider:

$$q = [0.6, 0.5, 0.4, 0.3].$$

The ideal binary document is 1111.

Suppose the database only contains:

Document A: 0111

Document B: 1000

If we greedily start with the first bit, the query prefers 1. We would therefore keep Document B and throw away Document A.

But their complete score values are:

$$S(A) = -0.6 + 0.5 + 0.4 + 0.3 = 0.6$$

while

$$S(B) = 0.6 - 0.5 - 0.4 - 0.3 = -0.6.$$

So Document A is actually better.

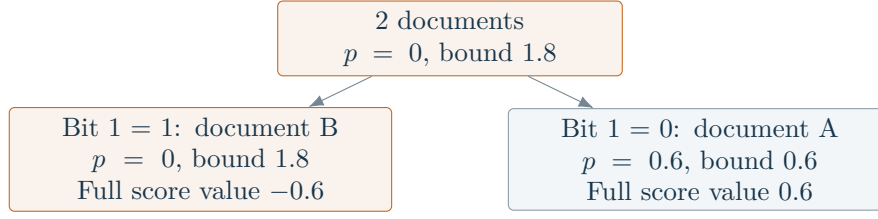
The problem is that matching one important bit does not guarantee that the remaining bits will also match.

Solution: branching

Instead of discarding the alternative, we keep both branches.

The branch matching the preferred bit receives no additional mismatch penalty. The alternative branch is still kept, but its penalty increases by $|q_i|$.

We then explore the branches in order of smallest accumulated penalty. This gives us a recursive search tree rather than one greedy path.



After the scoring operation for B, the other bound is $0.6 > -0.6$.
We must explore A; the first choice was not enough.

Figure 3: Keep the alternative until its bound on score values proves it cannot improve the result.

Stopping condition

Recall:

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|.$$

Here, Q is the sum of the absolute query values, so it is the score of the ideal binary document with every bit matching the query's preferred sign. For the query above, $Q = 0.6 + 0.5 + 0.4 + 0.3 = 1.8$, and it stays constant throughout the search.

If a branch has already accumulated mismatch penalty p , any document below it must have penalty at least p .

Therefore,

$$U_{\text{branch}} = Q - 2p$$

is an upper bound on the score value of every document inside that branch.

For example, suppose $Q = 2.5$ and we are looking for the top two results. We have already found 2.5 and 1.9, where 1.9 is the current top-2 threshold.

Now consider an unexplored branch with $p = 0.5$. Its absolute best possible score value is

$$2.5 - 2(0.5) = 1.5.$$

Since $1.5 < 1.9$, nothing inside that branch can enter our current top two. We can safely discard the entire subtree.

So:

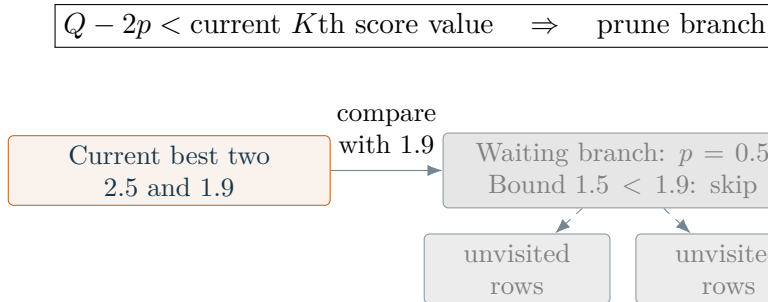


Figure 4: The whole gray subtree can be skipped because even its best possible score value is too low.

If the best remaining branch in the queue already fails this test, then every other branch is worse and the entire search can stop.

The resulting algorithm is approximately:

```

from heapq import heappop, heappush
from itertools import count

def bitplane_search(query, clusters, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    ideal_score = sum(abs(q) for q in query)
    sequence = count()
    queue = []
    for cluster in clusters:
        if cluster.full_mask:
            heappush(queue, (0, next(sequence), 0,
                             cluster.full_mask, cluster))

    best = TopK(k)
    while queue:
        penalty, _, depth, active, cluster = heappop(queue)
        upper_bound = ideal_score - 2 * penalty
        if best.full() and upper_bound < best.worst_score():
            break
        if active.bit_count() <= leaf_size or depth == len(query):
            for row_id, code in cluster.rows_in_mask(active):
                best.add(row_id, full_score(query, code))
            continue

        bit = order[depth]
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        matching = active & preferred
        # Keep the other half too; its remaining bits may be better.
        children = ((matching, 0), (active ^ matching, abs(query[bit])))
        for mask, extra_penalty in children:
            if mask:
                heappush(queue, (penalty + extra_penalty,
                                 next(sequence), depth + 1, mask, cluster))

    return best.results()

```

This is essentially the branch-and-bound Bitplane search used in the current experiments.

PROBLEM 2: Bitwise operations are cheap, but the bitmap itself can still be large

There is another issue.

A bitwise AND sounds almost free, but it is not literally one operation when the cluster contains thousands or millions of documents.

Suppose one cluster contains 6400 documents. With 64-bit machine words, one bitplane contains

$$6400/64 = 100$$

machine words.

So one split roughly requires us to loop over those 100 words.

Even worse, suppose after several splits only 80 documents remain. The physical mask can still be 100 words wide. So the next split may still need to process those same 100 words.

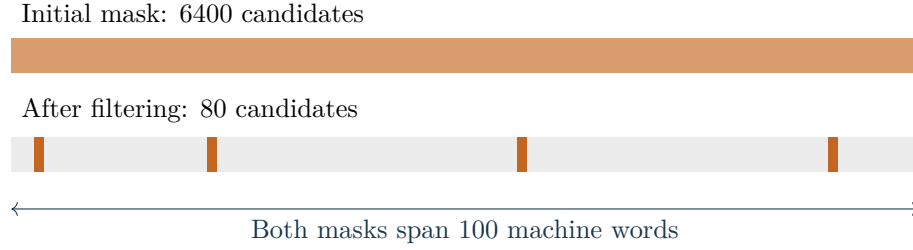


Figure 5: Fewer active document positions do not make this dense mask physically shorter.

Bitmap traversal cost. In general, for a cluster containing n documents, one dense bitplane operation costs roughly $O(\lceil n/64 \rceil)$ word operations. If we visit J split nodes, this becomes approximately $O(J \lceil n/64 \rceil)$, before even counting queue operations and the final query-document dot product operations. This was one of the important findings from the earlier experiments: reducing the number of documents that undergo a scoring operation does not automatically mean reducing total query time, because the bitmap traversal itself can become expensive.

Solution 1: Parameter tuning. This problem is technically something we can optimize. For example, we can tune cluster size during coarse routing; number of clusters probed; how deep we split; leaf size; when we switch back to ordinary query-document dot product operations. So Bitplanes is not fundamentally broken. The question is finding the point where scoring operations avoided are worth more than additional bitmap traversal. (This will be explored more in the experiments section later)

Solution 2: Other approach. However, thinking about this cost led me to a second idea. Bitplanes starts with a large set of documents and repeatedly narrows it down. What if we instead did the opposite? If Bitplanes can be thought of as bottom-up, then Backward Walk can be thought of as top-down: start from the most specific binary pattern we can describe and progressively broaden the search only when necessary.

1.3.2 Backward Walk

Backward Walk builds on the same observation:

Given a query, we already know what its ideal binary document should look like.

However, instead of starting with thousands of documents and filtering them using bitplanes, we try to start directly from a small bucket of documents that already looks similar to the query.

Suppose the binary query begins with 101101001011....

Using all 256 bits as a lookup key would be far too specific. With a large binary space, the probability that an exact 256-bit pattern actually exists in the database is extremely small.

Instead, suppose we choose a prefix length x . For example, the first 16 bits:

1011010010110101

During index construction, documents sharing this prefix are placed in the same bucket or contiguous range.

At query time, we directly look up this prefix. If it contains enough useful documents, we perform their scoring operations using the complete 256-bit scoring function. If it is empty, or contains too few candidates, we remove one constraint:

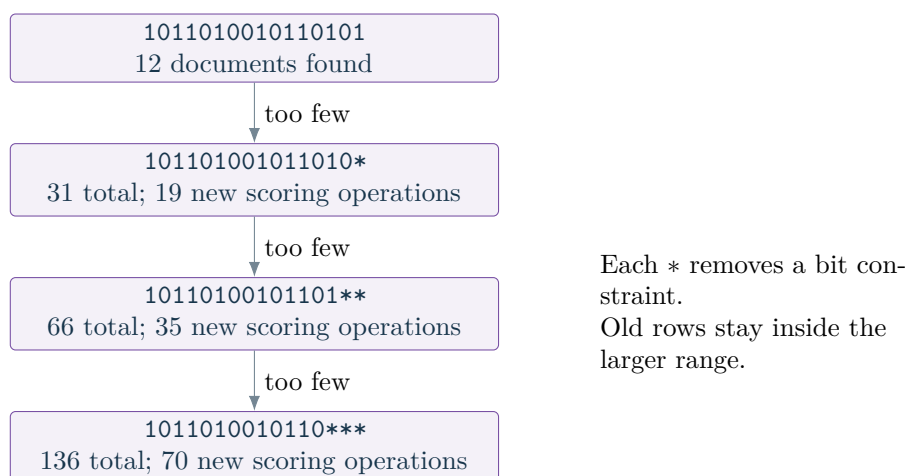


Figure 6: Illustrative counts: $12 + 19 + 35 + 70 = 136$ distinct query-document dot product operations, not $12 + 31 + 66 + 136$.

Every step moves to a larger candidate set.

This is why I call the method Backward Walk: instead of progressively adding constraints to a large candidate set, we begin from a very specific location and progressively walk backward toward more general prefixes.

Although obvious, I also want to quickly note that this bit prefix already has an exponential property: if the bits are independent and equally likely to be 0 or 1, each removed bit doubles the expected number of matching documents.

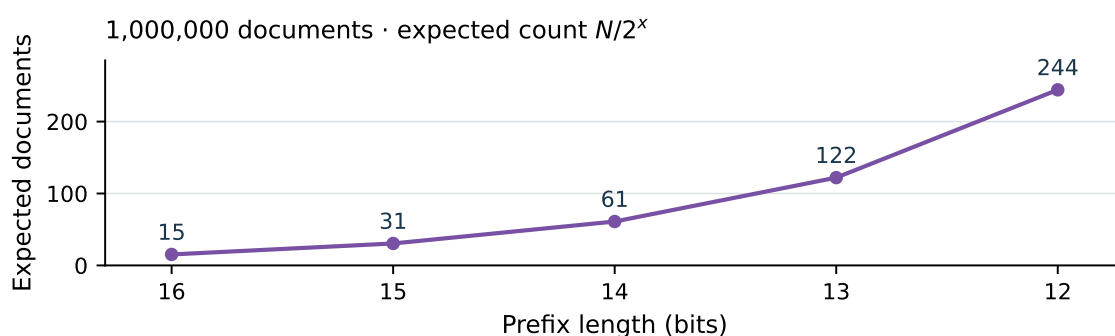


Figure 7: Expected counts for one global cluster; labels are rounded. This estimates rows, not recall.

With a trie or documents ordered by their prefix, the previous range is contained inside the parent range. We only need to process the newly introduced documents.

A simplified query algorithm becomes:

```
def backward_walk(query, clusters, k, start_depth, candidate_target):
    signs = tuple(q >= 0 for q in query)
    previous = [None] * len(clusters)
    best = TopK(k)
    scored = 0
    for depth in range(start_depth, -1, -1):
        for index, cluster in enumerate(clusters):
            left, right = cluster.prefix_range(signs[:depth])
            if previous[index] is None:
                new_ranges = [(left, right)]
            else:
                old_left, old_right = previous[index]
                # Only the new rows beside the old range need scoring operations.
                new_ranges = [(left, old_left), (old_right, right)]
            for start, stop in new_ranges:
                for row_id, code in cluster.rows_in_range(start, stop):
                    best.add(row_id, full_score(query, code))
                scored += stop - start
            previous[index] = (left, right)

        # Finish this depth in every cluster before testing the target.
        if candidate_target > 0 and scored >= candidate_target:
            break
    return best.results()
```

There is one important distinction here.

Stopping simply because we have collected, for example, 100 candidates is an approximate stopping rule. It does not mathematically prove that a better document does not exist outside the current prefix.

Therefore, in the experiments, recall still needs to be measured against the same exact reference results. An exact version would require a valid upper bound on the score values of documents outside the currently visited prefix.

Relationship to HNSW

At first glance, this approach may sound somewhat similar to HNSW because both methods use a hierarchy to progressively change the search space.

However, I think that the hierarchy is being used differently.

[top -> bottom] HNSW builds a proximity graph. During a query, it begins from sparse upper layers, navigates toward promising neighbors, and progressively descends toward the denser bottom layer [2].

[bottom -> top] Backward Walk does not need to navigate between neighboring document vectors. The query itself directly tells us where to begin:

query → binary prefix → exact prefix lookup

We then broaden that prefix only when necessary ("from the bottom, going to the top").

Potential advantage 1: cheaper index construction and updates

One possible advantage that I think might happen (and would like to investigate further) is the cost of adding new documents.

For HNSW, inserting a new vector generally requires searching the existing graph to find suitable neighbors and then creating or updating graph connections.

For Backward Walk, the conceptual insertion process is much simpler:

document → embedding → binary signs → prefix key → corresponding bucket

There is no nearest-neighbor graph that needs to be constructed for each new document. So Backward Walk may be attractive for a search index that changes frequently.

Future work: I think we can do further experiments and treat this as a hypothesis to verify, because the actual insertion cost still depends on how the buckets, ranges and on-disk/in-memory index are implemented.

Potential advantage 2: Matryoshka prefixes give the relaxation some "meaning"

In the method C, I also think that there is also a useful connection with Matryoshka embeddings. A Matryoshka model is trained so that shorter prefixes of the embedding remain useful representations [3].

Therefore, moving from, for example, 16 bits → 15 bits → 14 bits is not simply dropping arbitrary dimensions from an ordinary embedding. We are progressively moving to shorter prefixes of a representation that was specifically trained to retain useful information at multiple prefix lengths.

So, this gives us a "reasonable reason" to do Method C, as shorter prefixes remain meaningful, however, of course, the actual recall of each prefix still has to be measured.

Future work: one optimization that we can do in the future is to find the best balance between number of prefix bits (which is connected to latency) and the recall performance.

2 Preliminary calculations

[Note: this part is quite long, if you are interested in the [experiment result](#), you can skip this section]

Before running the experiments, let's first break down what's the time/memory complexity for each method.

to recap:

- Method A performs a "query-document dot product" operation for every document inside the opened clusters.
- Method B avoids some "query-document dot product" operations by filtering with Bitplanes first.
- Method C avoids starting with a large document set by looking up a specific prefix, then broadening only when needed.

So for each method, I mainly want to understand two things:

- What additional work do Methods B and C introduce?
- How many "query-document dot product" operations can Methods B and C potentially save?

Let's use this as variables:

Symbol	Meaning
N	Total number of documents
C	Total number of clusters
P	Number of clusters opened for one query (P is for "probes")
d	Number of binary dimensions
K	Number of results we want to return
n_c	Number of documents inside cluster c
F_A, F_B, F_C	Number of documents that undergo a "query-document dot product" operation under A, B and C

If the clusters are roughly balanced, one cluster contains about

$$n \approx \frac{N}{C}$$

documents.

So if we open P clusters, we expose roughly

$$\frac{PN}{C}$$

documents.

2.1 Query-time work

Let's try to imagine this using example. Suppose one opened cluster contains three documents:

ID	Binary document	Full score value
0	10100	0.1
1	11001	2.5
2	11000	1.9

The query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the previous section, we know that the ideal binary document is

11001

and suppose we want the best

$$K = 2$$

documents.

For this example:

$$N = 3, \quad C = P = 1, \quad d = 5.$$

*note: the actual experiments later contain hundreds of thousands or millions of documents.

2.1.1 Method A: scan the opened clusters

Method A is the easiest one to understand.

Once cluster selection (prefilter) has decided which clusters to open, **for every document inside those clusters, we will do a "query-document dot product" operation.**

Step 1: how many "query-document dot product" operations do we need?

In our example, the only opened cluster contains:

[ID 0, ID 1, ID 2]

so:

$$F_A = 3.$$

More generally, let O be the set of clusters opened for one query. Then:

$$F_A = \sum_{c \in O} n_c$$

is the number of documents for which A performs "query-document dot product" operations.

If the clusters are roughly balanced:

$$F_A \approx \frac{PN}{C}.$$

So for A, this is already the most important number: **how many documents did cluster selection leave for us to search?**

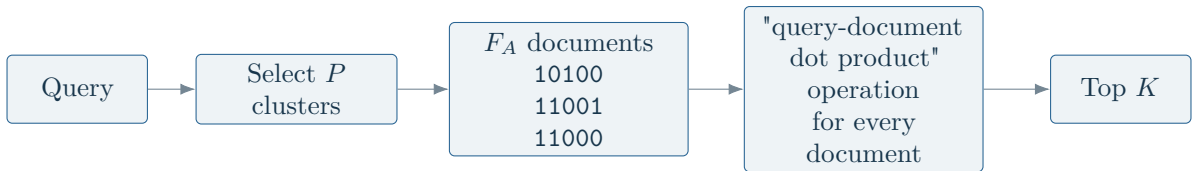


Figure 8: Method A performs no additional filtering after cluster selection. Every document inside the opened clusters undergoes a "query-document dot product" operation.

Step 2: what does one "query-document dot product" operation cost?

The scorer used here follows the four-sign lookup-table approach described in Exa's article.

Instead of multiplying every query value with every document bit individually, the query is divided into groups (or subvectors) of four.

For our five-dimensional example:

$$\begin{bmatrix} 0.7, & 0.5, & -0.4, & -0.6 \\ 0.3, & 0, & 0, & 0 \end{bmatrix}$$

A four-bit group has

$$2^4 = 16$$

possible binary patterns.

So this example needs

$$2 \times 16 = 32$$

precomputed table entries.

In general, the number of groups is

$$G = \left\lceil \frac{d}{4} \right\rceil.$$

One "query-document dot product" operation therefore reads G table entries.

For our example:

$$G = 2.$$

Three documents therefore require:

$$3 \times 2 = 6$$

table lookups.

In general, the scoring work grows with

$$F_A G.$$

Since G grows with d ,

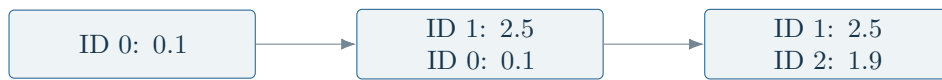
$$T_{\text{score},A} = O(F_A d).$$

For a fixed binary dimension, this is basically linear in the number of documents for which we perform "query-document dot product" operations.

Step 3: keep only the best K

As score values are produced, we only need to keep the best K .

For our example:



Using a bounded heap, maintaining the best K results takes at most approximately

$$O(F_A \log(K + 1)).$$

Total cost for A

Let T_{base} contain the work before "query-document dot product" operations, such as cluster selection and lookup-table preparation.

Let a_A be the average measured time needed to fully process one document, including its "query-document dot product" operation and the top- K update.

Then:

$$T_A \approx T_{\text{base}} + F_A a_A.$$

For roughly balanced clusters:

$$T_A \approx T_{\text{base}} + \frac{PN}{C} a_A.$$

The main tradeoff is straightforward.

Using more clusters usually makes each cluster smaller, so each opened cluster contains fewer documents. However, we may then need to open more clusters to maintain the same recall.

TL;DR, Method A: A's cost is mainly determined by how many documents cluster selection leaves behind. Fewer opened documents means fewer "query-document dot product" operations, but opening too few clusters can hurt recall.

2.1.2 Method B: Bitplanes

Bitplanes try to reduce the number of documents that undergo a "query-document dot product" operation.

Instead of immediately performing a "query-document dot product" operation for every document inside the opened clusters, B first uses Bitplanes to filter the documents.

So B has two main costs:

Bitplane work + "query-document dot product" operations for the documents that survive

The whole idea only makes sense if the "query-document dot product" operations we avoid are more expensive than the Bitplane work required to avoid them.

Step 1: inspect the important query bits first

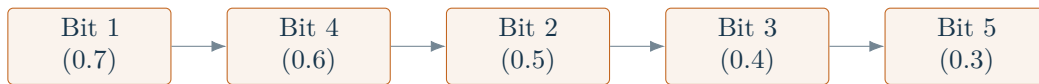
B uses the same cluster selection and the same lookup table as A.

The additional step is to order the query bits by decreasing $|q_i|$.

For

$$q = [0.7, 0.5, -0.4, -0.6, 0.3],$$

the order is:



We inspect the larger values first because mismatching them hurts the score value more.

Sorting these positions costs

$$O(d \log d).$$

We also calculate the ideal score value:

$$Q = \sum_i |q_i| = 2.5.$$

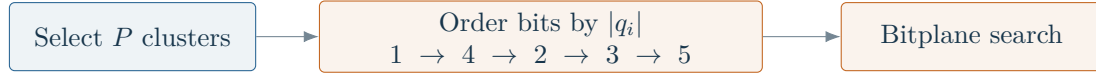


Figure 9: Bitplanes keep the same cluster-selection stage as A, but add query-dependent filtering before the full scorer.

Step 2: represent the documents using a mask

Inside one cluster, B keeps a bitmap showing which document positions are still active.

If cluster c contains n_c documents, its mask needs

$$W_c = \left\lceil \frac{n_c}{64} \right\rceil$$

64-bit machine words.

Our three-document example fits inside one word.

The starting mask is:

111

which means that all three documents are still active.

Step 3: split using the query bits

The first query position is bit 1, and the query prefers 1.

All three documents match:

$$111 \text{ AND } 111 = 111$$

so nothing is removed.

The next position is bit 4. The query prefers 0, and again all three documents match.

So the mask is still:

111.

The next position is bit 2.

Its Bitplane is:

011.

Therefore:

$$111 \text{ AND } 011 = 011$$

The preferred branch now contains IDs 1 and 2.

The alternative branch is:

100

which contains ID 0.

Because ID 0 disagrees with the query at bit 2, this branch has mismatch penalty:

$$p = |q_2| = 0.5.$$

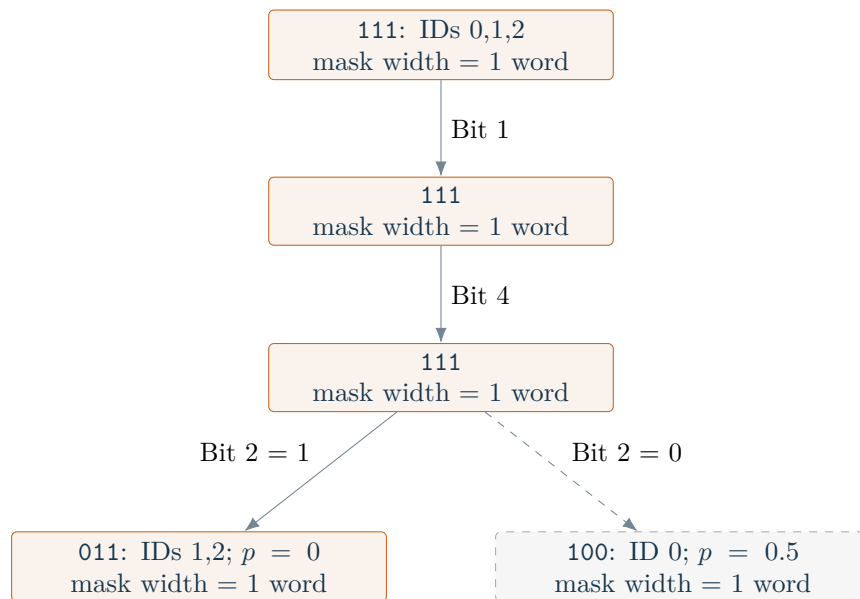


Figure 10: The first two bit tests remove no documents. The third separates ID 0 into another branch, but that branch is not discarded yet.

Step 4: why Bitplane filtering is not $O(1)$, it's not free! :(

At first, a bitwise AND seems like it's $O(1)$.

But actually it's not. Example -> If one cluster contains:

6,400

documents.

Then, for every one mask we need:

$$6400/64 = 100$$

machine words.

So one split does not perform one AND, as we need to loop across roughly 100 word positions.

There is another important detail: **the mask does not physically get shorter when fewer documents remain active.**

For example, suppose only 80 documents survive after several splits. But in this case, the mask can still be 100 words wide.

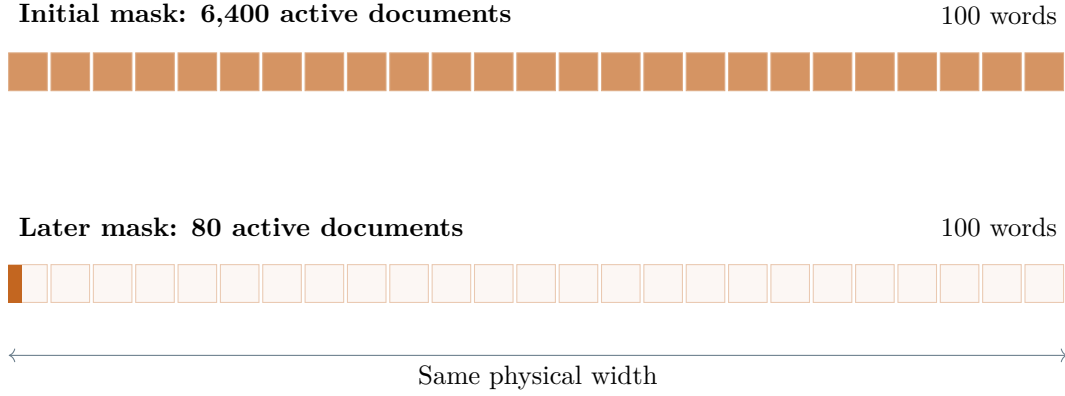


Figure 11: The number of active documents can become much smaller while the physical bitmap stays the same size.

Step 5: count the Bitplane work

Let J_c be the number of times B splits a mask in cluster c .

Each split scans W_c words.

So the total amount of split-mask work is:

$$V_{\text{split}} = \sum_{c \in O} J_c W_c.$$

For roughly balanced clusters, if B performs an average of S splits per opened cluster:

$$V_{\text{split}} \approx PS \left\lceil \frac{N}{64C} \right\rceil.$$

Step 6: eventually stop splitting and perform "query-document dot product" operations

After a branch contains few enough documents, we treat this as a **leaf**, and in this case, method B will perform "query-document dot product" operations for the documents inside it.

Suppose our leaf size is two.

The preferred branch:

011

contains exactly IDs 1 and 2.

So B performs these "query-document dot product" operations:

$$\begin{aligned} \text{ID 1} &\rightarrow 2.5 \\ \text{ID 2} &\rightarrow 1.9 \end{aligned}$$

Therefore:

$$F_B = 2.$$

*in this case, if we do a comparison, method A would have performed three "query-document dot product" operations.

another little cost: we still need to read leaf masks

Before B can extract the surviving document positions, it still has to scan the leaf mask. Let H_c be the number of leaf masks read in cluster c .

$$V_{\text{leaf}} = \sum_{c \in O} H_c W_c.$$

Step 7: the un-explored branch that is still in the queue

Remember when we consider 'branching' to catch all cases because we are worried that we might miss the ideal case due to branching? Imagine that: ID 0 is still waiting to be explored.

We can calculate the known mismatch penalty, which is:

$$p = 0.5.$$

However, we can take a look that the best possible score value inside that branch is:

$$Q - 2p = 2.5 - 2(0.5) = 1.5.$$

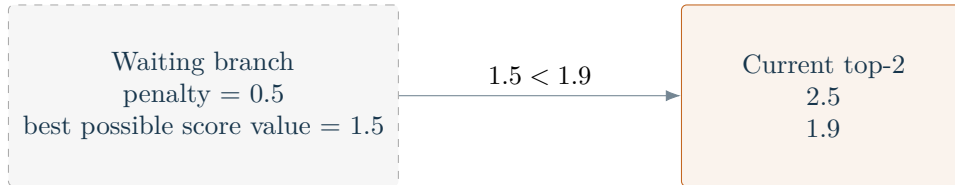
Meanwhile, our current top two score values are already:

$$2.5, \quad 1.9.$$

Since:

$$1.5 < 1.9,$$

We can make conclusion: nothing inside the waiting branch can enter the current top two. So the good news is that, we can safely skip it!



→ skip branch

Figure 12: Once the best possible score value of a branch falls below the current top- K threshold, the whole branch can be skipped.

Total cost for B

B therefore pays for:

- ordering the query bits;
- reading split masks;
- reading leaf masks;
- maintaining the waiting branches;
- performing F_B "query-document dot product" operations.

Let:

Symbol	Meaning
b_s	Average time for one split-mask word
b_l	Average time for one leaf-mask word
T_{order}	Ordering query bits and preparing query weights
T_{queue}	Managing branches, masks and the priority queue
a_B	Average time to fully process one surviving document

Then:

$$T_B \approx T_{\text{base}} + T_{\text{order}} + b_s V_{\text{split}} + b_l V_{\text{leaf}} + F_B a_B + T_{\text{queue}}.$$

The key condition is:

$$\text{"query-document dot product" work avoided} > \text{Bitplane work added.}$$

A smaller leaf size may reduce F_B , because B keeps filtering longer. However, that also means more splits and more bitmap work.

A larger node budget can also improve recall, but again it allows B to search more branches.

TL;DR, Bitplanes: It is fast when a few Bitplane splits can filter out most documents before the "query-document dot products" operations. If the search needs many splits, or the masks remain wide, the bitmap work grows and can cancel out the savings from scoring fewer documents.

2.1.3 Method C: Backward Walk

Now let's look at the second idea.

Backward Walk approaches the problem from almost the opposite direction.

Method A starts with every document inside the opened clusters.

Method B also starts with all of them, but progressively removes documents.

Backward Walk instead asks:

What if we start from the most specific binary prefix first, and only broaden the search when we need more documents?

So C mainly pays for two things:

- looking up different prefixes;
- performing "query-document dot product" operations for the documents exposed by those prefixes.

Step 1: store the documents in prefix order

Using the same example, let's sort the three documents by their binary key:

Position	Binary key	ID
0	10100	0
1	11000	2
2	11001	1

Meanwhile, the ideal query pattern is 11001.

Because the rows are sorted, documents with the same prefix will appear next to each other.

That means we can use binary search to quickly find the range corresponding to a particular prefix.

Step 2: start from the most specific prefix

Let's first use all five bits: 11001.

Two binary searches find the beginning and the end of this range.

In our example, the result is $[2, 3)$, which contains only ID 1.

So we perform one "query-document dot product" operation: $\text{ID } 1 \rightarrow 2.5$.

However, remember that we want the top two results. Right now we only have one document.

So unfortunately, we cannot stop yet. We need to broaden the prefix.

Step 3: remove one constraint

We now go from 11001 to 1100*.

The * means that we no longer care whether the last bit is 0 or 1.

The new range becomes $[1, 3)$, which contains:

- ID 2: 11000;
- ID 1: 11001.

Now, ID 1 was already scored at the previous depth.

So the good news is that we do **not** need to score it again.

The only new document is ID 2: $\text{ID } 2 \rightarrow 1.9$.

Now we have two results.

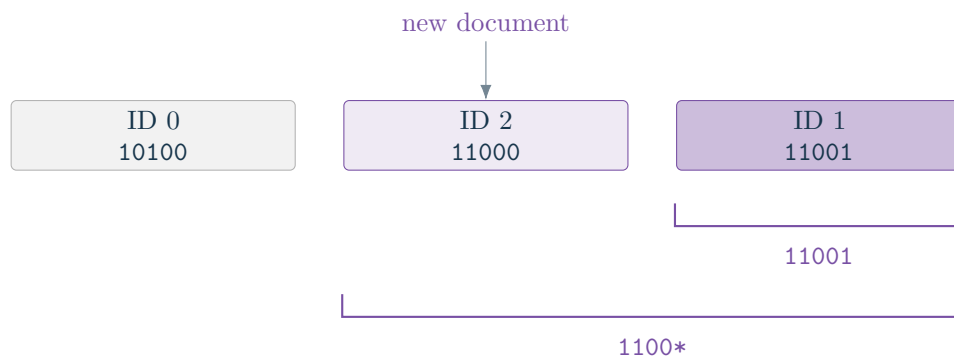


Figure 13: The broader prefix contains the previous range. ID 1 is already scored, so only the newly included document, ID 2, needs a "query-document dot product" operation.

Step 4: how much prefix lookup work do we need?

Suppose we start at prefix depth x , and eventually stop at depth ℓ .

Then the number of depths we visit is:

$$L = x - \ell + 1.$$

At every positive depth, we need two binary searches for every opened cluster:

- one to find where the prefix starts;
- one to find where it ends.

So if we open P clusters and visit L_+ positive prefix depths, we perform $2PL_+$ binary searches.

Depth zero uses the known whole-cluster range and needs no binary search.

Each binary search over cluster c costs roughly:

$$O(\log(n_c + 1)).$$

So the total lookup work is approximately:

$$2L_+ \sum_{c \in O} \log_2(n_c + 1).$$

Again, the formula looks more complicated than the idea.

The idea is simply:

Backward Walk cost grows with how many prefix depths we visit and how many clusters we search at each depth.

Step 5: how many documents do we eventually score?

Let F_C be the total number of distinct documents that become visible before we stop.

Because each broader prefix contains the previous prefix, a document that was already scored does not need another "query-document dot product" operation.

So C's document-scoring work is simply:

$$F_C a_C.$$

Here a_C is the average time for a "query-document dot product" operation and its top- K update.

This means the main question for Backward Walk is actually:

How broad does the prefix need to become before we have enough useful documents?

If we can stop while the prefix is still very specific, F_C can be very small.

If we need to keep broadening all the way to the root, F_C eventually becomes all documents in the opened clusters.

Step 6: how quickly does the prefix grow?

Let's use a simple model just to build intuition.

Assume, temporarily, that every bit is independent and has a 50% chance of being 0 or 1.

Then the probability that a random document matches an ℓ -bit prefix is:

$$2^{-\ell}.$$

So if we have n documents, the expected number matching that prefix is:

$$\frac{n}{2^\ell}.$$

For a collection of one million documents:

Prefix depth	Expected documents
16	about 15
15	about 31
14	about 61
13	about 122
12	about 244

So something interesting happens here.
We only remove **one bit** each time, but the number of matching documents approximately **doubles**.

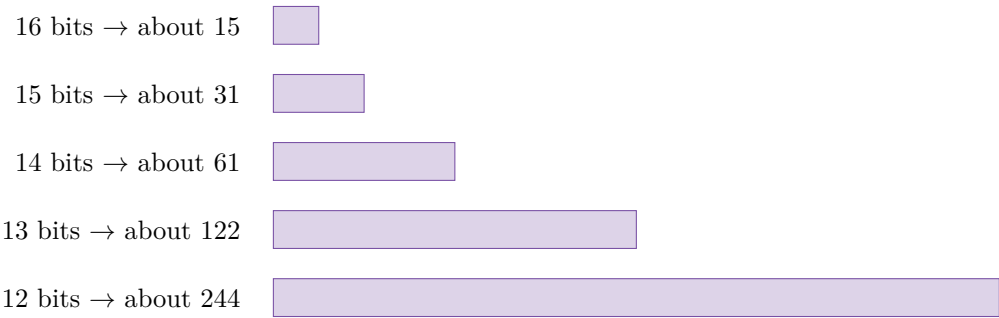


Figure 14: Under the independent 50/50-bit model, removing one prefix bit doubles the expected matching count. The labels are rounded; they describe document counts, not recall.

However, we should remember that this is only an intuition.
Real embedding bits can be correlated, and once the documents have already been divided into clusters, the distribution can be even further from a perfect 50/50 split.
Most importantly:

This only estimates how many documents we might find. It does not tell us whether those documents are actually the correct top results.

Step 7: when should Backward Walk stop?

There are two ways to stop.
Approximate stopping
The simplest option is to stop after we have found enough documents.
For example:

16 bits	→ 12 documents
15 bits	→ 31 documents
14 bits	→ 66 documents
13 bits	→ 136 documents

If our target is 100 documents, we could stop at 13 bits.
This is simple and cheap. However, there is one problem.
Finding 100 documents does not prove that the best unseen document is worse than the ones we found.
So this stopping rule is approximate, and we need to measure its recall experimentally.
Exact stopping

Fortunately, we can also use the scoring equation to derive a safe bound.

Suppose we have already searched all documents matching the first ℓ query signs inside the opened clusters.

Any unseen document in those clusters must disagree with the query on at least one of those first ℓ positions.

The cheapest possible mismatch is:

$$m_\ell = \min_{1 \leq i \leq \ell} |q_i|.$$

Therefore, the best possible score of an unseen document is:

$$U_\ell = Q - 2m_\ell.$$

Meanwhile, suppose we have already kept K documents and the worst document currently inside our top- K has score τ .

If:

$$\tau > U_\ell,$$

then even the **best possible unseen document** cannot enter our top- K .

So the good news is that we can stop safely. The implementation also keeps its tie-breaking and floating-point rounding checks.

If this never happens, we can keep broadening the prefix until depth zero.

At depth zero, we have exposed every document inside the opened clusters, so Backward Walk becomes equivalent to Method A locally.

Total cost for Backward Walk

Backward Walk therefore pays for:

- building the query prefix;
- performing prefix lookups;
- updating the current prefix ranges;
- performing "query-document dot product" operations for F_C exposed documents.

We can summarize this as:

$$T_C \approx T_{\text{base}} + T_{\text{key}} + 2L + t_b \sum_{c \in O} \log_2(n_c + 1) \\ + F_C a_C + T_{\text{ranges}}.$$

T_{key} covers the query key and optional bound preparation; t_b is time per binary-search comparison; T_{ranges} covers range updates and stopping checks.

Again, the exact symbols are not the most important part.

The important condition is:

"query-document dot product" work avoided > prefix lookup work added

If C can stop at a fairly deep prefix, then $F_C \ll F_A$, and we may save a lot of scoring work.

However, if C has to broaden the prefix all the way to depth zero, $F_C = F_A$, then C ends up doing all of A's "query-document dot product" operations **plus** the additional prefix lookups.

TL;DR, Backward Walk: It is fast when a specific prefix already contains the documents we need. Otherwise, it must keep relaxing the prefix and expanding the searched range; if this goes too far, Backward Walk approaches a full scan while also paying the extra binary-search cost for each new prefix range.

2.2 Memory complexity

So far, we have only talked about query time.

However, B and C also need additional index structures, so we should also check how much memory they require.

There are two kinds of memory that matter here:

- **persistent index memory**, which remains loaded between queries;
- **temporary query memory**, which only exists while one query is running.

2.2.1 Method A memory

Method A stores:

- the packed binary document;
- the document ID;
- the cluster/routing information.

Using 64-bit words, one binary document needs

$$8 \left\lceil \frac{d}{64} \right\rceil$$

bytes. The ID adds another 8 bytes.

So the document storage is:

$$M_{A,\text{rows}} = N \left(8 \left\lceil \frac{d}{64} \right\rceil + 8 \right).$$

During a query, A also keeps the query values, the lookup table and the current top- K .

2.2.2 Method B memory

B also uses what A has been used, and additional two more things. First, it stores the Bitplanes.

For each cluster, every binary dimension needs one bitmap spanning all document positions in that cluster.

Therefore:

$$M_{\text{planes}} = 8d \sum_{c=1}^C W_c.$$

So in a rough sense, B stores the document bits twice:

- once arranged by document;
- once arranged as Bitplanes.

The second extra cost is temporary query memory.

Remember that branching means we can have several masks waiting in the queue at the same time.

Each one still has the full physical width of its cluster.

If $A_c(t)$ masks belonging to cluster c are alive at time t , then the raw mask memory is:

$$Q_{\text{masks}} = 8 \max_t \sum_c A_c(t) W_c.$$

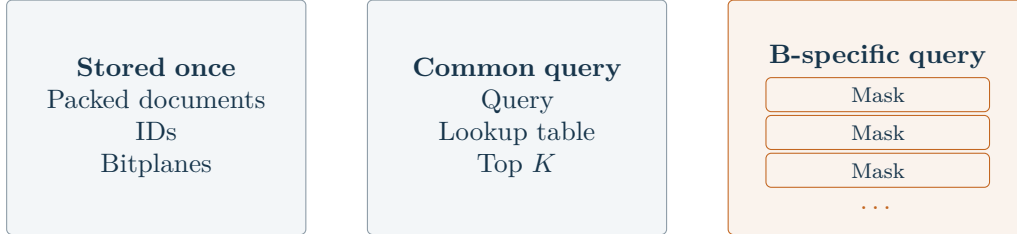


Figure 15: B stores Bitplanes alongside its packed documents. During a query, several full-width masks may also remain alive in the branch queue.

2.2.3 Method C memory

C also keeps the normal packed documents and IDs.

The current implementation additionally stores one 32-bit prefix key for every document. So:

$$M_{\text{prefix}} = 4N.$$

The important part is that we do **not** create a new copy of the documents for every prefix depth. For example, 11001, 1100*, 110** all refer to different ranges of the **same sorted document array**.

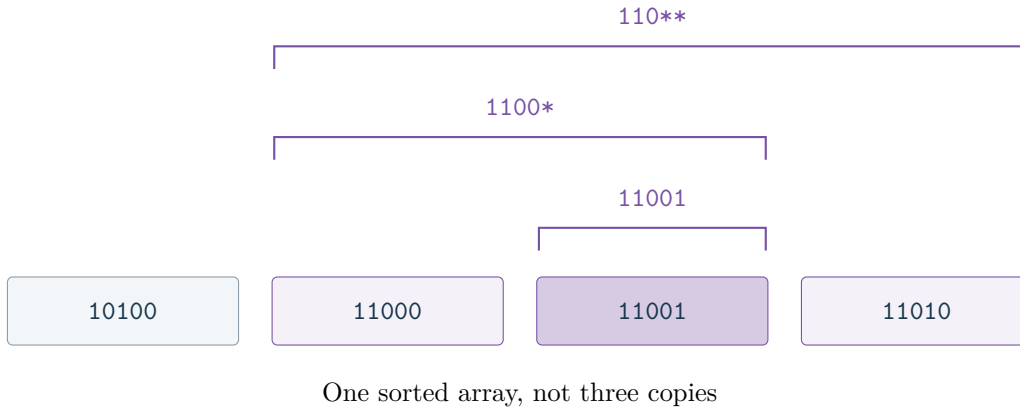


Figure 16: In this illustrative array, three prefix ranges refer to the same stored rows. Broadening the prefix does not create a new document list.

During the query, C mainly needs to remember the previous range for every opened cluster.

So compared with B, method C's temporary search state is relatively small.

2.3 Recall

Now we know roughly how much work each method performs.

However, none of those savings matter if we stop too early and return the wrong documents.

So we also need to understand where recall can be lost.

For query q :

- G_q is the exact global top- K ;
- $O_{a,q}$ is the result returned by method a .

Average recall is:

$$R_a = \frac{\sum_q |G_q \cap O_{a,q}|}{n_q K}.$$

Here n_q is the number of queries. The reference and returned results use the same score and document-ID tie rule.

For example:

Reference: [1, 2]
Returned: [1, 0]

means we recovered one of two: 50%.

Where can the correct document disappear?

There are actually two places.

First, cluster selection can fail to open the cluster containing the correct document.

Second, even if the correct document is inside an opened cluster, B or C may stop their local search before reaching it.

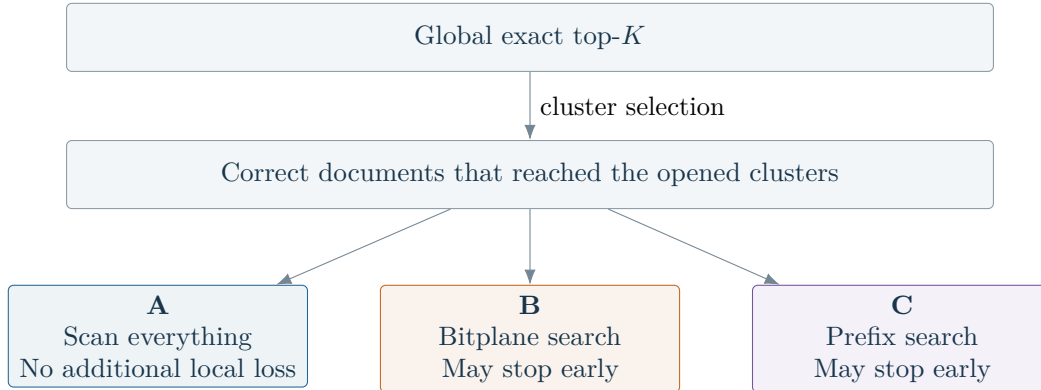


Figure 17: Cluster selection can lose a correct document before local search begins. B and C may also lose a document by stopping their local search early.

2.3.1 Recall for Method A

Let g_q be the number of global top- K documents whose clusters were opened.

Because A scores every document inside those clusters, all g_q of those documents are recovered locally. So:

$$R_{A,q} = \frac{g_q}{K}.$$

In other words:

A can lose a correct document because cluster selection missed it, but once the document is inside an opened cluster, A will not lose it during local search.

2.3.2 Recall for Bitplanes

Suppose g_q correct documents reached the opened clusters, but B only recovers some fraction $r_{B,q}$ of them.

Then:

$$R_{B,q} = \frac{g_q}{K} r_{B,q}.$$

If $g_q = 0$, recall is zero.

For example, imagine:

- 8 of the global top 10 documents are inside the opened clusters;
- B recovers 6 of those 8.

Then:

$$\text{Recall} = \frac{8}{10} \times \frac{6}{8} = 60\%.$$

If B performs a complete branch-and-bound search using valid score bounds, then it eventually recovers all of the correct documents inside the opened clusters.

However, if we impose a finite node budget, B may stop early.

TL;DR, B recall: B can lose recall in two places: cluster selection can miss the document before B starts, and a limited Bitplane search can miss a document even after its cluster has been opened.

2.3.3 Recall for Backward Walk

For Backward Walk, let $D_{q,\ell}$ be all documents exposed when the search stops at prefix depth ℓ .

Then:

$$R_{C,q} = \frac{|G_q \cap D_{q,\ell}|}{K}.$$

The important thing here is that **candidate count and recall are not the same thing**.

Finding 100 documents does not mean that we found the correct 100 documents.

Consider:

$$q = [0.1, 0.9].$$

The ideal binary pattern is 11 and suppose the only documents are 10 and 01.

Their scores are:

$$S(10) = -0.8, \quad S(01) = 0.8.$$

If we search prefix 1*, we find 10.

However, the better document is 01, which is outside that prefix.

So even though we found one candidate, we did not find the best result.

That is why stopping after “enough documents” is only an approximate rule.

C is exact locally only when:

1. we broaden all the way to depth zero and expose every document inside the opened clusters;
or
2. the score bound proves that none of the unseen documents can beat the current top- K .

TL;DR, C recall: A large prefix range does not automatically mean high recall. Backward Walk can safely stop only when it has either searched every document or mathematically proved that the unseen documents cannot win.

2.4 Summary

The whole calculation section can be summarized with one simple idea:

Method	What it does
A	Score every document inside the opened clusters
B	Spend Bitplane work to avoid some "query-document dot product"
C	Spend prefix-lookup work to avoid some "query-document dot product"

So:

A just pays for "query-document dot product". B and C deliberately add extra search work, hoping that this lets them avoid enough "query-document dot product" to make the total query faster.

The experiment in the next section will tell us whether that actually happens on real embeddings.

3 Experiments

So far, we have only looked at the three methods mathematically.

The calculations tell us what *could* make Bitplanes or Backward Walk faster, but the actual result depends on the documents, the queries, and how much work each method ends up doing.

So the experiment is simple:

For the same search problem and the same recall requirement, what is the fastest configuration I can find for A, B and C?

Each method is allowed to choose the parameters that make sense for it.

Method A can choose its cluster count and how many clusters to open.

Method B can choose those same cluster settings, together with its leaf size and node budget.

Method C can choose the cluster settings, starting prefix depth and stopping rule.

The documents, queries, binary scoring function and correct answers remain unchanged.

This is important because B and C do not necessarily work best with the same cluster layout as A. For example, Bitplanes may be able to open a much larger group of documents and then remove most of them before the "query-document dot product" operation.

3.1 Setup

I use two collections:

Collection	Embedding model	Documents	Queries	Binary dimensions
MS MARCO	Nomic v1.5	1,000,000	1,000	64, 256, 768
Quora	Qwen3 0.6B	522,931	1,000	32, 256, 1024

The document and query embeddings are generated before the search experiment begins.

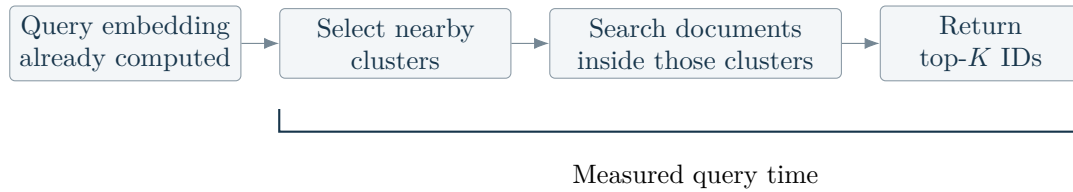


Figure 18: The query embedding is already available when timing begins. Query time includes cluster selection, search inside those clusters and returned IDs.

The timer therefore measures only the part of the pipeline that I am trying to improve: Embedding generation and the later full-vector reranking stage are not included.

For one comparison, A, B and C use:

- the same documents;
- the same 1,000 queries;
- the same binary dimension;
- the same scoring function;
- the same requested result count K ;
- the same exact reference answers.

The idea is simply: **same search problem, different way of searching it.**

Changing the binary dimension is treated as a different comparison because, for example, a 32-bit binary vector and a 256-bit binary vector do not necessarily rank the documents in the same order.

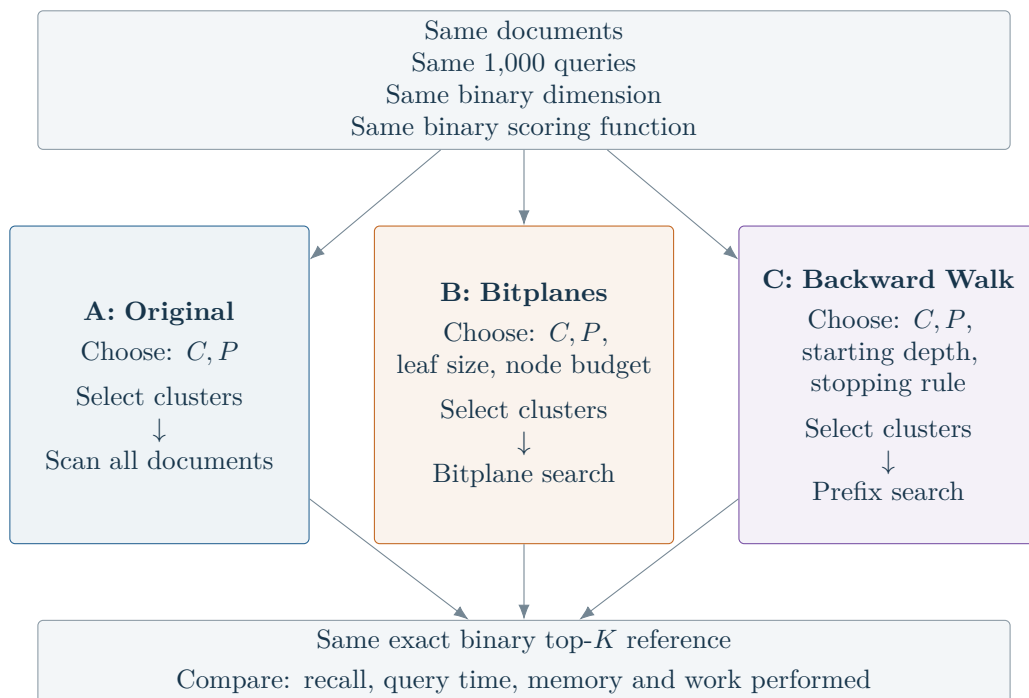


Figure 19: A, B and C search the same documents and queries using the same binary scoring function. Each method can choose its own index and search parameters; the fastest configuration meeting the same recall requirement is compared.

3.2 How the comparison works

Before testing approximate search, I first calculate the exact answer for every query.

I do this by performing a scoring operation for every document in the collection using the same binary scoring function used by A, B and C.

Suppose the exact top five document IDs are [12, 7, 91, 4, 30] and one method returns [12, 7, 91, 4, 85].

Four of the five correct IDs were recovered, so:

$$\text{Recall@5} = 80\%.$$

I repeat this over all 1,000 queries.

Therefore, in this document, if it's written as **99% recall**, we mean that the method recovers 99% of the exact binary top- K document IDs on average.

Choosing the parameters

For every method, we try multiple configurations.

For A:

- number of clusters C ;
- number of opened clusters P .

For B:

- C ;
- P ;
- leaf size;
- node budget.

For C:

- C ;
- P ;
- starting prefix depth;
- stopping rule.

For a given recall target, such as 80%, 90%, 95%, 99%, I discard configurations that do not reach that recall.

Among the remaining configurations, I keep the fastest one for each method.

So the comparison is:

3.3 Main result

Let's first look at Qwen with 32-bit binary documents while returning one result: $K = 1$.

This is the setting where the difference between the three search strategies is the clearest.

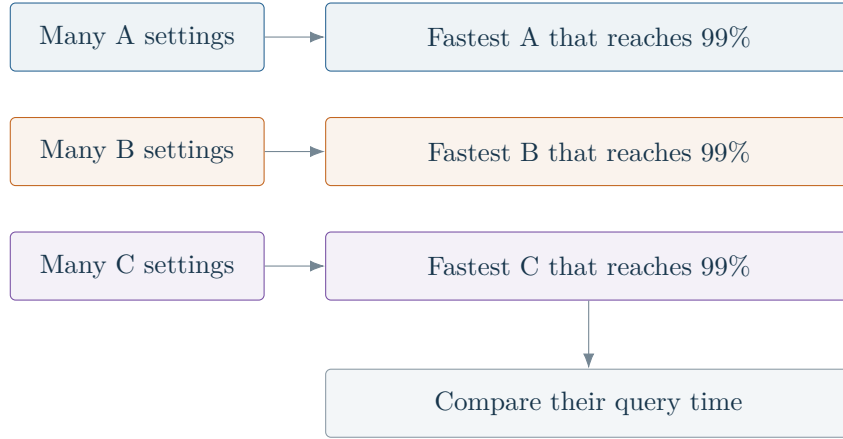


Figure 20: At each recall target, compare the fastest tested configuration for each method that meets the requirement. Measurements below the target remain in the saved results, but cannot win that comparison.

Required recall	A: Original	B: Bitplanes	C: Backward Walk
80%	0.0308 ms	0.0295 ms	0.0318 ms
90%	0.0498 ms	0.0432 ms	0.0497 ms
95%	0.0805 ms	0.0520 ms	0.0799 ms
99%	0.1948 ms	0.0809 ms	0.1688 ms

At the 99% recall target, Bitplanes take 0.0809 ms compared with 0.1948 ms for A.

So in this setting, B is approximately 2.4× faster than the original method.

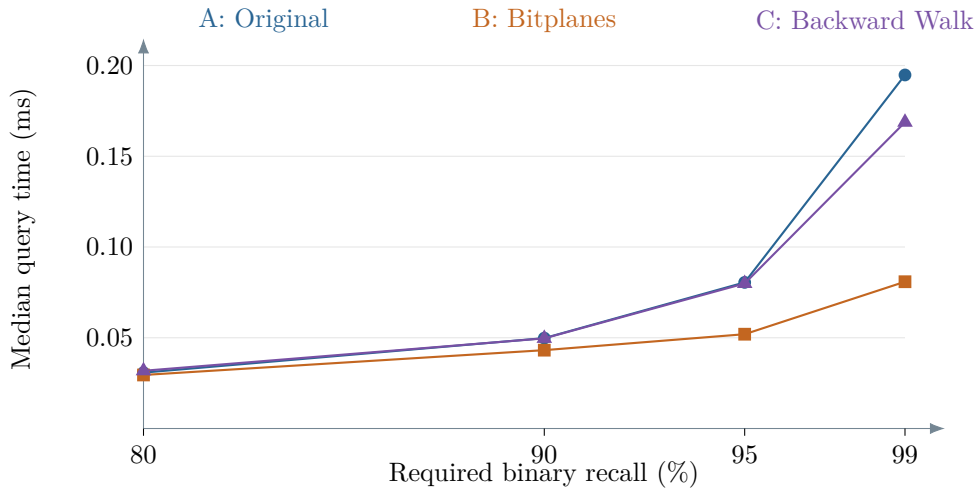


Figure 21: Query time for Qwen32 with $K = 1$. Each point shows the repeated median for the fastest tested configuration selected at the required binary recall.

3.4 Why is Bitplanes faster?

let's look at the recall 99% settings chosen by each method.

Method	Clusters	Clusters opened	Search setting	Recall	Time (ms)
A	4096	180	Scan	99.0%	0.1948
B	16	9	Leaf 128, budget 8192	99.2%	0.0809
C	1024	78	Start depth 4, exact stop	99.0%	0.1688

The first interesting thing is that the methods choose very different cluster layouts.

Now we can also observe how many documents reach the search stage and how many of them undergo a "query-document dot product" operation:

Method	Documents inside opened clusters	Scoring operations
A	24,181	24,181
B	295,031	818
C	41,147	26,153

We can observe the following:

Method A

A uses very selective cluster selection:

select clusters $\rightarrow$ 24,181 documents $\rightarrow$ 24,181 scoring operations.

Method B

B does almost the opposite:

select much larger clusters $\rightarrow$ 295,031 documents
 $\downarrow$ Bitplanes
 818 "query-document dot product" operations.

B therefore performs scoring operations for only

$$818/295031 \approx 0.28\%$$

of the documents inside its opened clusters.

This is exactly the tradeoff described in Section 2. B performs extra bitmap work, but in this case that work removes so many "query-document dot product" operations that the complete query becomes faster.

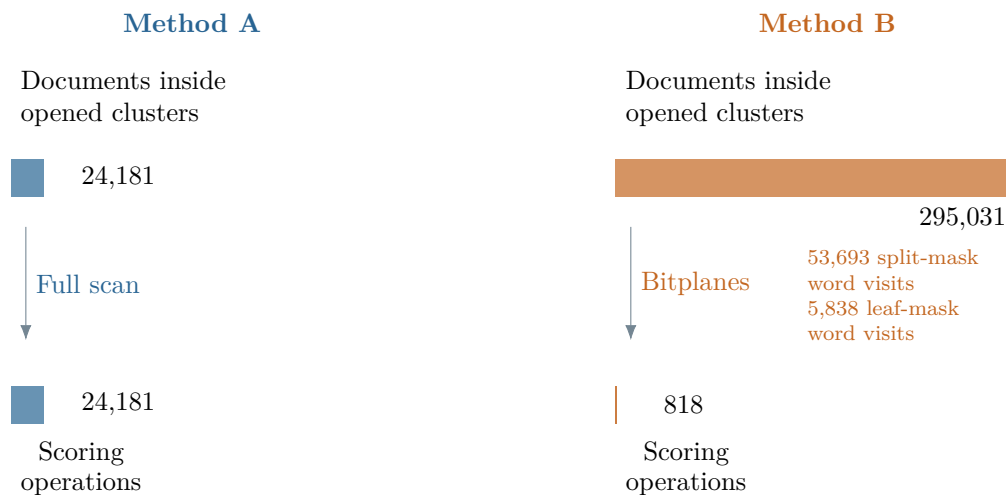


Figure 22: At the selected 99% setting, A opens a small set of documents and performs scoring operations for all of them. B opens a much larger set but uses Bitplanes to reduce roughly 295,000 documents to only 818 "query-document dot product" operations.

What about C?

Backward Walk also removes some "query-document dot product" operations:



Figure 23: C reduces the number of "query-document dot product" operations, but exposes a larger fraction of its selected documents than B does.

However, the reduction is much smaller than B's. C also needs to perform prefix lookups while progressively broadening the query prefix.

At this setting, C visits around 3.5 prefix depths and performs around 495 boundary searches per query.

Its final time is 0.1688 ms, which is better than A in this particular comparison, but still much slower than B.

So the three methods are effectively making three different choices:

A	Reduce documents heavily during cluster selection, then perform a scoring operation for every document
B	Use coarse cluster selection, then aggressively prune with Bitplanes
C	Use medium cluster selection, then progressively broaden a prefix

3.5 Does this advantage always happen?

TLDR: Not really. The previous result asks for only one final result: $K = 1$. Now let's say, we increase the number of requested results to $K = 100$.

At the 99% binary recall target:

Representation	A	B	C
Nomic 64	1.915 ms	2.071 ms	1.875 ms
Nomic 256	13.453 ms	13.866 ms	14.645 ms
Nomic 768	45.088 ms	45.217 ms	45.196 ms
Qwen 32	0.400 ms	0.439 ms	0.404 ms
Qwen 256	3.564 ms	3.721 ms	4.039 ms
Qwen 1024	22.863 ms	23.098 ms	22.930 ms

In these selected settings, B performs no useful splitting and C starts from depth zero. In other words, both methods effectively fall back toward the original scan.

C's slightly lower median on Nomic64 is a scan-like timing difference; it does not come from avoiding "query-document dot product" operations.

This also makes intuitive sense. When we only need one result, B can search the most promising branches and safely ignore many alternatives.

When we need 100 results, many more branches can still contain one of the required documents. B therefore has to search much more of the tree.

Eventually, the cost of pruning is no longer worth it.

The Bitplane advantage therefore appears strongest when K is small enough that many branches can be safely ignored.

This is a measured result for the tested settings, not a guarantee that B can never help for larger K .

3.6 Summary

For Qwen32 with one requested result, Bitplanes are clearly faster in the tested search setup.

At 99% binary recall:

- A takes 0.1948 ms;
- B takes 0.0809 ms;
- C takes 0.1688 ms.

B is therefore about $2.4\times$ faster than A.

Why?.

A selects a relatively small number of documents and then performs scoring operations for all of them.

B selects a much larger number of documents, but its Bitplane search reduces roughly 295,000 opened documents to only around 818 "query-document dot product" operations.

In this setting, the "query-document dot product" work avoided is large enough to repay the extra bitmap work.

However, this does not happen everywhere. When the result count increases to $K = 100$, B and C become much closer to ordinary scanning and the advantage disappears in the tested settings.

Backward Walk shows the same general tradeoff, but in the tested data it does not reduce the scoring work as aggressively as Bitplanes.

4 Limitations and next steps

Environment might be different.

In this experiment, all three methods are tested using the same CPU implementation and the same "query-document dot product" code. This gives us a fair comparison inside this report, but a production system may behave differently. For example, Exa may use a more optimized scorer, different SIMD kernels, or a different memory layout. If "query-document dot product" becomes much cheaper, then avoiding a scoring operation also becomes less valuable. One useful extension would be to test the same methods in a more production-like environment, or against an implementation closer to the system described by Exa.

Bitplanes are effective for small K , but the benefit becomes less predictable as K grows.

In the experiments, Bitplanes perform very well when we only need a small number of results. The reason is that many branches can be ignored once we already have a strong top result. However, when K becomes larger, more branches may still contain one of the required documents, so B has to keep exploring and the extra bitmap work can start to dominate. One possible direction for future work is to predict, from the query and the requested K , whether Bitplanes are likely to be useful before running them.

Backward Walk still needs further investigation.

The current version works best when a fairly specific query prefix already contains useful documents. If that prefix is too restrictive, the method has to keep broadening it, and eventually it may expose a large part of the opened clusters. A useful next step would be to study better ways of choosing the starting prefix, the order of the prefix bits, or when to stop broadening.

Real retrieval quality still needs to be investigated.

The recall measured in this report is based on the exact ranking of the binary index being tested. For example, 99% recall with a 32-bit representation means recovering 99% of the exact 32-bit binary results. It does not necessarily mean that those results match the original full embedding or the documents that are most relevant to a user. A more complete evaluation should also compare against the full embedding and, where possible, human relevance labels.

Other operations, such as index updates, also matter in production.

The experiments here focus on query-time search, but a real search system also needs to insert, delete and update documents while continuing to serve queries. This is especially interesting for Backward Walk, because its prefix-based index may be simpler to update than a graph-based structure, but that has not been measured yet. Future work should therefore look at both query latency and index-maintenance cost.

5 Conclusion

This report started from a simple observation: once we have a floating-point query, we already know the ideal binary document and how costly each bit mismatch would be.

From that idea, I explored two alternatives to the original scan. Bitplanes try to remove unlikely documents before scoring them, while Backward Walk tries to begin from a small prefix range and only broaden the search when needed.

The experiments show that the extra search structure can be useful, but only in the right setting. The clearest result is Bitplanes with a small requested result count, where the method can avoid a very large number of document scoring operations and become substantially faster than the original scan. When more results are required, that advantage becomes much smaller because more of the search space still needs to be explored.

Backward Walk shows a similar tradeoff from the opposite direction. It can be very cheap when a useful prefix already isolates the right documents, but it becomes less attractive when the prefix has to be broadened too far.

So the main takeaway is not that one method should always replace the others. It is that **the best search strategy depends on how much work we can avoid compared with the extra work required to avoid it.**

That gives a useful direction for future work: instead of choosing one search method for every query, we may be able to choose between scanning, Bitplanes and prefix search depending on the query, the requested K , and the structure of the opened clusters.

References

- [1] The Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS, 2022. <https://arxiv.org/abs/2205.13147>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model card, accessed 13 September 2026. The saved manifest identifies the exact model revision. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.

- [5] Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. <https://arxiv.org/abs/2506.05176>. Encoding recipe: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [6] P. Bajaj et al. *MS MARCO: A Human Generated MACHine Reading COMprehension Dataset*. 2016; revised 2018. <https://arxiv.org/abs/1611.09268>. Passage-ranking data: <https://github.com/microsoft/MSMARCO-Passage-Ranking>.
- [7] N. Thakur et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2104.08663>. Dataset catalog: <https://github.com/beir-cellar/beir>.
- [8] M. Bawa, T. Condie and P. Ganesan. *LSH Forest: Self-Tuning Indexes for Similarity Search*. WWW, 2005, pp. 651 to 660. <https://www.cs.princeton.edu/courses/archive/spring06/cos592/bib/LSHForest-bawa05.pdf>.
- [9] M. Aumüller, T. Christiani, R. Pagh and M. Vesterli. *PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors*. ESA, 2019. <https://doi.org/10.4230/LIPIcs.ESA.2019.10>.